



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek II.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2025. november 6.

Tartalom I

- 1 Súlyozott gráf
 - Definíciók
 - Gyakorlatok
- 2 Minimális feszítőfa
 - Definíciók
 - Általános módszer minimális feszítőfa építésére
- 3 Kruskal
 - Leírás
 - Gyakorlat
 - Algoritmus
- 4 Diszjunkt halmaz adatstruktúra
- 5 Prim
 - Leírás
 - Struktogram

Tartalom II

- Gyakorlat

6 Piros-kék színezés

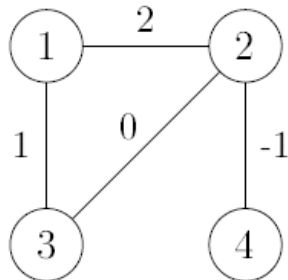
7 Gyakorlat

Súlyozott gráf

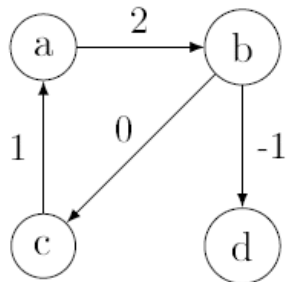
Definíció: Súlyozott gráf egy $G = (V, E)$ gráf a $w : E \rightarrow \mathbb{R}$ súlyfüggvénnyel. Egy $(u, v) \in E$ élhez tartozó $w(u, v)$ értéket az él **súlyának**, **hosszának** vagy **költségének** nevezzük (ezek azonos jelentésűek).

Egy **út súlya/hossza/költsége** az út éleinek súlyösszege.

Súlyozott gráfok grafikus és szöveges reprezentációja:



$1 - 2, 2 ; 3, 1.$
 $2 - 3, 0 ; 4, -1.$



$a \rightarrow b, 2.$
 $b \rightarrow c, 0 ; d, -1.$
 $c \rightarrow a, 1.$

Grafikus repr.: az élek a súlyukkal vannak feliratozva.

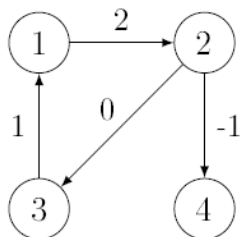
Szöveges repr.: szomszéd-súly párok listája u_i, w_{u_i} , ahol u_i a szomszéd, w_{u_i} pedig az oda vezető él súlya.

Súlyozott gráfok szomszédsági mátrixos reprezentációja:

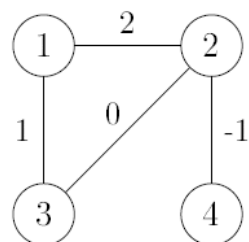
$A/1 : \mathbb{R}_\infty[n, n]$ (a mátrix valós számokat vagy ∞ -t tartalmaz)

Ha a csúcshalmaz $V = \{v_1, v_2, \dots, v_n\}$, akkor

- $A[i, j] = w(v_i, v_j) \iff (v_i, v_j) \in E$
- $A[i, i] = 0$
- $A[i, j] = \infty \iff (v_i, v_j) \notin E \text{ és } i \neq j$



A	1	2	3	4
1	0	2	∞	∞
2	∞	0	0	-1
3	1	∞	0	∞
4	∞	∞	∞	0



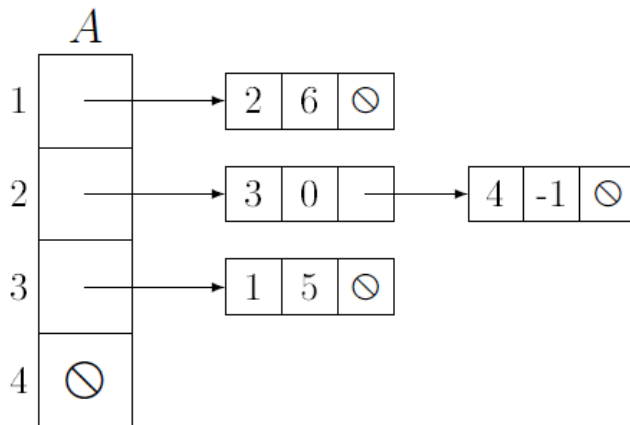
A	1	2	3	4
1	0	2	1	∞
2	2	0	0	-1
3	1	0	0	∞
4	∞	-1	∞	0

Megjegyzés: a szomszédsági mátrix főátlójában mindig 0 áll (mind irányított, mind irányítatlan esetben), mivel nincsenek hurokélek. További 0-k is előfordulhatnak, ha van 0 súlyú él, de ha nincs él, akkor ∞ szerepel.

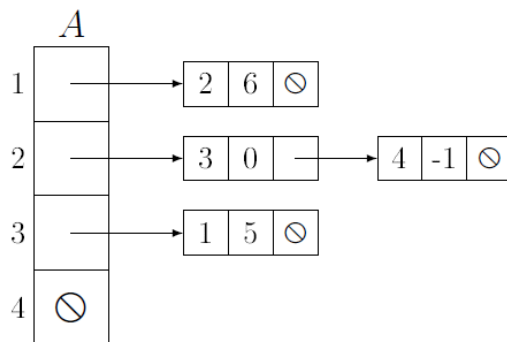
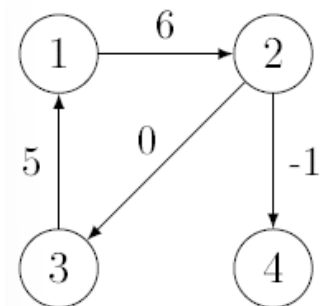
Megjegyzés: egy irányítatlan gráf szomszédsági mátrixa mindig szimmetrikus.

Szomszédsági listás reprezentáció: $A/1$ egy mutatótömb
 $A : \text{Edge}^*[n]$, ahol az **Edge** típus a következő:

<i>Edge</i>
$+v : \mathbb{N}$
$+w : \mathbb{R}$
$+next : \text{Edge}^*$



Példa:



$A[1] \rightarrow v = 2 ; A[1] \rightarrow w = 6 ; A[1] \rightarrow next = \otimes$

$A[2] \rightarrow v = 3 ; A[2] \rightarrow w = 0 ; A[2] \rightarrow next \rightarrow v = 4$
 $; A[2] \rightarrow next \rightarrow w = -1 ; A[2] \rightarrow next \rightarrow next = \otimes$

$A[3] \rightarrow v = 1 ; A[3] \rightarrow w = 5 ; A[3] \rightarrow next = \otimes$

$A[4] = \otimes$

Gráfrepresentációk tárigénye:

- szomszédsági mátrix: $\Theta(n^2)$
- szomszédsági lista: $\Theta(n + m)$

Súlyozott gráfok absztrakt osztálya:

$\mathcal{G}_w$
$+ V : \mathcal{V}\{\}$
$+ E : \mathcal{E}\{\} // E \subseteq V \times V \setminus \{(u, u) : u \in V\}$
$+ w : E \rightarrow \mathbb{R} // \text{weights of edges}$

Gyakorlatok:

- 1 Legyen G egy súlyozott, irányítatlan gráf szomszédsági listás reprezentációban. Adj meg egy algoritmust, amely kiszámítja G szomszédsági mátrixát. (A láncolt listák elemei növekvő sorrendben vannak $p \rightarrow v$ szerint.) Mi az algoritmus műveletigénye?
- 2 Legyen G egy súlyozott, irányított gráf szomszédsági mátrixos reprezentációban. Adj meg egy algoritmust, amely előállítja G szomszédsági listás reprezentációját úgy, hogy a listák elemei növekvő sorrendben legyenek $p \rightarrow v$ szerint. Mi az algoritmus műveletigénye?

Minimális feszítőfa

Minimális feszítőfa

2.1. Definíció. A $G = (V, E)$ irányítatlan gráf feszítő erdeje a $T = (V, F)$ gráf, ha $F \subseteq E$, valamint T (irányítatlan) erdő (azaz T olyan irányítatlan gráf, aminek mindegyik komponense irányítatlan fa, a fák páronként diszjunktak, és együtt éppen lefedik a G csúcshalmazát).

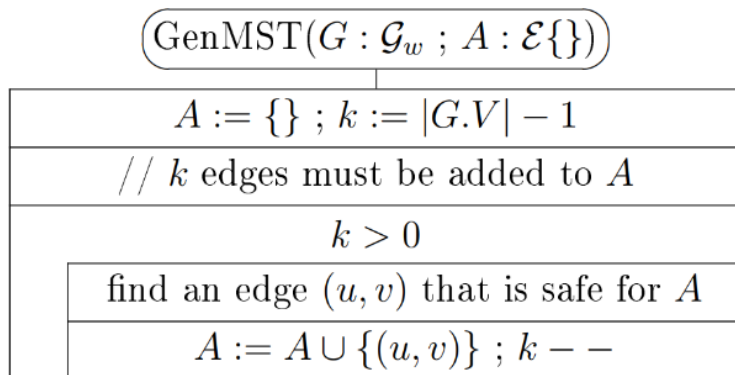
2.2. Definíció. A $G = (V, E)$ irányítatlan, összefüggő gráf feszítőfája a $T = (V, F)$ gráf, ha $F \subseteq E$, és T (irányítatlan) fa.

2.3. Definíció. Amennyiben $G = (V, E)$ élsúlyozott gráf (fa, erdő stb.) a $w : E \rightarrow \mathbb{R}$ súlyfüggvénnyel, akkor a G súlya az élei súlyainak összege:

$$w(G) = \sum_{e \in E} w(e)$$

2.4. Definíció. A G irányítatlan, összefüggő, élsúlyozott gráf minimális feszítőfája (minimum spanning tree: MST) T , ha T a G feszítőfája, és G bármely T' feszítőfájára $w(T) \leq w(T')$.

Az alábbi általános módszer az $A = \{\}$ üres élhalmazból indul, és ezt úgy bővíti újabb és újabb élekkel, hogy A végig a G összefüggő, irányítatlan, él-súlyozott gráf valamelyik minimális feszítőfája élhalmazának a részhalmaza marad: Éppen az így választott éleket nevezzük az A élhalmazra nézve *biztonságosnak* (*safe for A*). Amikor az élek száma eléri a $|G.V|-1$ értéket, az A szükségszerűen feszítőfa, és így minimális feszítőfa is lesz.



2.5. Definíció. *Tegyük fel, hogy $G = (V, E)$ élsúlyozott, irányítatlan, összefüggő gráf, és $A \subseteq a$ G valamelyik minimális feszítőfája élhalmazának! Ekkor az $(u, v) \in E$ él biztonságosan hozzávehető az A élhalmazhoz (safe for A), ha $(u, v) \notin A$ és $A \cup \{(u, v)\} \subseteq a$ G valamelyik (az előzővel nem okvetlenül egyező) minimális feszítőfája élhalmazának.*

2.6. Következmény. *Tegyük fel, hogy $G = (V, E)$ élsúlyozott, irányítatlan, összefüggő gráf! Ha egy kezdetben üres A élhalmazt újabb és újabb biztonságosan hozzávehető éllel bővítünk, akkor $|G.V|-1$ bővítés után éppen a G egyik minimális feszítőfáját kapjuk meg.*

A kérdés most az, hogyan tudunk mindig biztonságos élet választani az A élhalmazhoz. Ehhez lesz szükségünk az alábbi fogalmakra és tételre.

Hogyan találhatunk *biztonságos* éleket?

2.7. Definíció. Ha $G = (V, E)$ gráf és $\{\} \subsetneq S \subsetneq V$, akkor a G gráfon $(S, V \setminus S)$ egy vágás.

2.8. Definíció. $G = (V, E)$ gráfon az $(u, v) \in E$ él keresztezi az $(S, V \setminus S)$ vágást, ha $(u \in S \wedge v \in V \setminus S) \vee (u \in V \setminus S \wedge v \in S)$.

2.9. Definíció. $G = (V, E)$ élsúlyozott gráfon az $(u, v) \in E$ könnyű él az $(S, V \setminus S)$ vágásban, ha (u, v) keresztezi a vágást, és $\forall (p, q)$, a vágást keresztező éltre $w(u, v) \leq w(p, q)$.

2.10. Definíció. A $G = (V, E)$ gráfban az $A \subseteq E$ élhalmazt elkerüli az $(S, V \setminus S)$ vágás, ha az A egyetlen éle sem keresztezi a vágást.

2.11. Tétel. *Ha a $G = (V, E)$ irányítatlan, összefüggő, élsúlyozott gráfon*
(1) A részhalmaza a G valamelyik minimális feszítőfája élhalmazának,
(2) az $(S, V \setminus S)$ vágás elkerüli az A élhalmazt, és
(3) az $(u, v) \in E$ könnyű él az $(S, V \setminus S)$ vágásban,
 $\implies$ az (u, v) él biztonságosan hozzávehető az A élhalmazhoz.

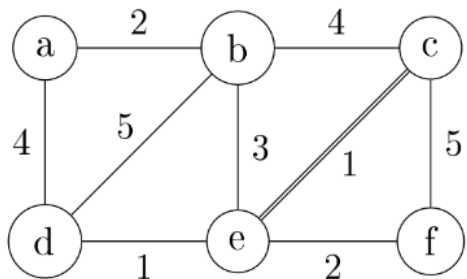
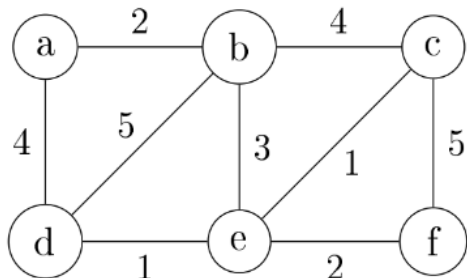
Ennek a tételnek a bizonyítása a feltöltött jegyzetben található.

Általános módszer minimális feszítőfa építésére

Általános módszer minimális feszítőfa építésére

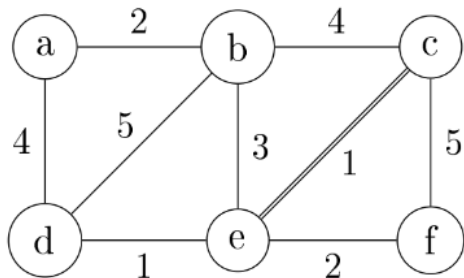
Algoritmus:

- Kezdjünk egy $A = \{\}$ halmazzal.
- Folytasd, amíg $|A| = n - 1$.
 - Keressünk egy vágást, amely tiszteletben tartja A -t.
 - Keressünk egy könnyű élt, amely ezt a vágást metszi.
 - Add ezt az élt A -hoz.

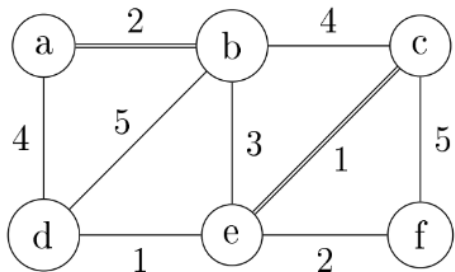


Kezdetben $A = \{\}$, így bármelyik vágás jó. Válasszuk pl. az $(\{a, b, c\}, \{d, e, f\})$ vágást! Ebben (c, e) a könnyű, tehát biztonságos él. Vegyük hozzá A -hoz!

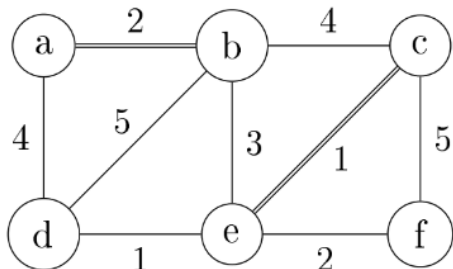
Most $A = \{(c, e)\}$. Ezt elkerüli pl. az $(\{a, f\}, \{b, c, d, e\})$ vágás, amiben (a, b) és (e, f) a könnyű, tehát biztonságos élek. Válasszuk pl. (a, b) -t!



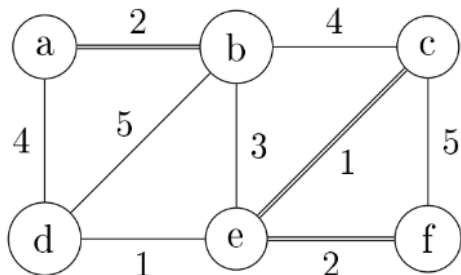
Most $A = \{(c, e)\}$. Ezt elkerüli pl. az $(\{a, f\}, \{b, c, d, e\})$ vágás, amiben (a, b) és (e, f) a könnyű, tehát biztonságos élek. Válasszuk pl. (a, b) -t!



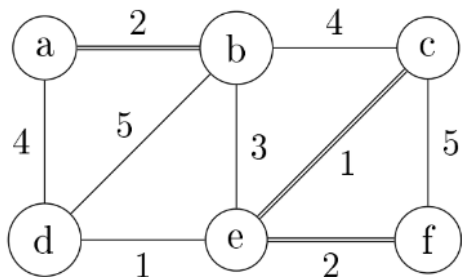
Most $A = \{(a, b), (c, e)\}$. Ezt elkerüli pl. az $(\{a, b, c, d, e\}, \{f\})$ vágás, amiben (e, f) a könnyű, tehát biztonságos él.



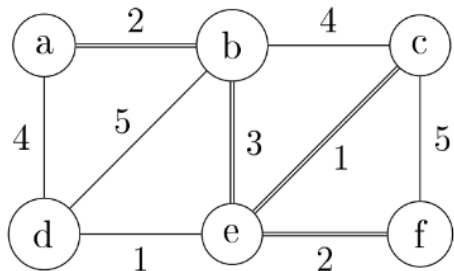
Most $A = \{(a, b), (c, e)\}$. Ezt elkerüli pl. az $(\{a, b, c, d, e\}, \{f\})$ vágás, amiben (e, f) a könnyű, tehát biztonságos él.



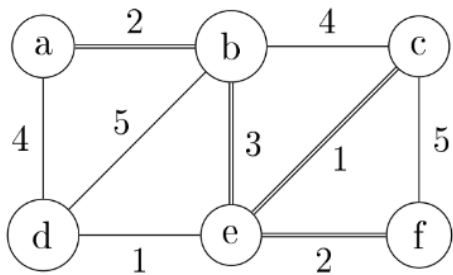
Most $A = \{(a, b), (c, e), (e, f)\}$. Ezt elkerüli pl. az $(\{a, b\}, \{c, d, e, f\})$ vágás, amiben (b, e) a könnyű, tehát biztonságos él.



Most $A = \{(a, b), (c, e), (e, f)\}$.
Ezt elkerüli pl. az
 $(\{a, b\}, \{c, d, e, f\})$ vágás, ami-
ben (b, e) a könnyű, tehát biz-
tonságos él.



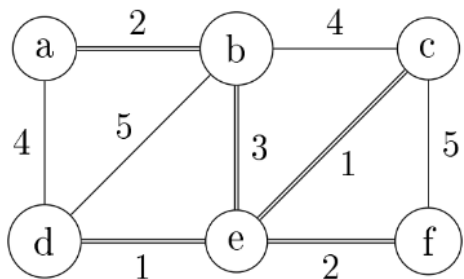
$A = \{(a, b), (b, e), (c, e), (e, f)\}$.
Ezt egyedül az
 $(\{a, b, c, e, f\}, \{d\})$ vágás kerüli
el, amiben (d, e) a könnyű, tehát
biztonságos él.



$$A = \{(a, b), (b, e), (c, e), (e, f)\}.$$

Ezt egyedül az

$(\{a, b, c, e, f\}, \{d\})$ vágás kerüli el, amiben (d, e) a könnyű, tehát biztonságos él.



$A = \{(a, b), (b, e), (c, e), (d, e), (e, f)\}.$
 $|A| = 5 = |G.V| - 1$, tehát A egy minimális feszítőfa (MST) élhalmaza.

Kruskal

Vizualizáció:

- 1 <https://visualgo.net/en/mst>
- 2 <https://www.cs.usfca.edu/galles/visualization/Kruskal.html>

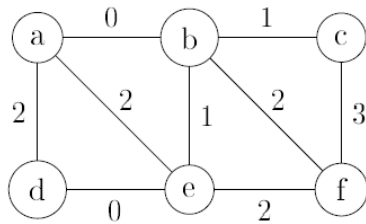
Kruskal algoritmus

- Rendezzük a gráf éleit növekvő sorrendbe a súlyuk szerint.
- Építsük fel az A élhalmazt (az MST éleit) úgy, hogy az éleket egyenként adjuk hozzá. Üres halmazból indulunk $A = \{\}$, és az éleket növekvő sorrendben vizsgáljuk.
- Ha egy él nem hoz létre kört az A halmaz éleivel, akkor biztonságos, és hozzáadható A -hoz. Ha kört hoz létre, elvetjük, és a következő élre lépünk.
- Az algoritmus futása közben az A élhalmazzal rendelkező csúcsok halmaza erdőként értelmezhető:
 - Kezdetben n különálló fánk van, mindegyik egyetlen csúcsból áll.
 - Minden új él hozzáadásával két fát összekapcsolunk, így nagyobb fa jön létre.
 - Minden él, amely két különböző fát köt össze, biztonságos. Miért?
 - Legyen (u, v) az éppen vizsgált él, és X az u -t tartalmazó fa csúcshalmaza. Ekkor X egy vágás, és (u, v) ennek a vágásnak a „következő”.

Kruskal algoritmusának helyessége

- Egy él, amely egy fa két csúcsát köti össze, kört hoz létre, ezért elvetjük.
- Minden él, amely két különböző fát köt össze, nem hoz létre kört. Az ilyen él biztonságos az A halmazhoz. Miért?
- Legyen (u, v) az éppen vizsgált él, és X az u -t tartalmazó fa csúcshalmaza. Ekkor $(X, V \setminus X)$ egy vágás, amely tiszteletben tartja A -t, és (u, v) ennek a vágásnak a könnyű éle. Így alkalmazható a 2.12-es tétel.
- Miért (u, v) a könnyű él a $(X, V \setminus X)$ vágásban? Mert csak olyan élek metszenek ezt a vágást, amelyeket még nem vizsgáltunk, és ezek mindegyikének súlya legalább $w((u, v))$ (mivel az éleket növekvő sorrendben vizsgáljuk).

2.2 Algorithm of Kruskal



$a - b, 0$; $d, 2$; $e, 2$.

$b - c, 1$; $e, 1$; $f, 2$.

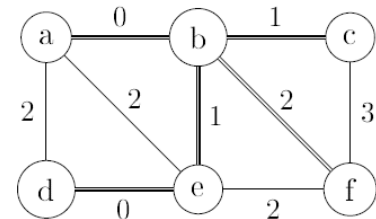
$c - f, 3$.

$d - e, 0$.

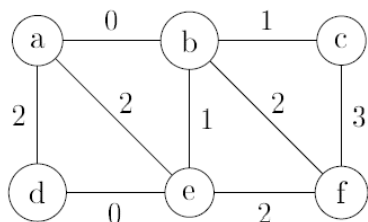
$e - f, 2$.

edges in increasing order:
 (a, b) , (d, e) , (b, c) , (b, e) ,
 (a, d) , (a, e) , (b, f) , (e, f) ,
 (c, f)

Components	edge	its weight	is it safe?
a, b, c, d, e, f	(a,b)	0	
	(d,e)	0	
	(b,c)	1	
	(b,e)	1	
	(a,d)	2	
	(a,e)	2	
	(b,f)	2	

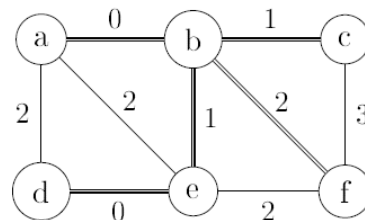


2.2 Algorithm of Kruskal

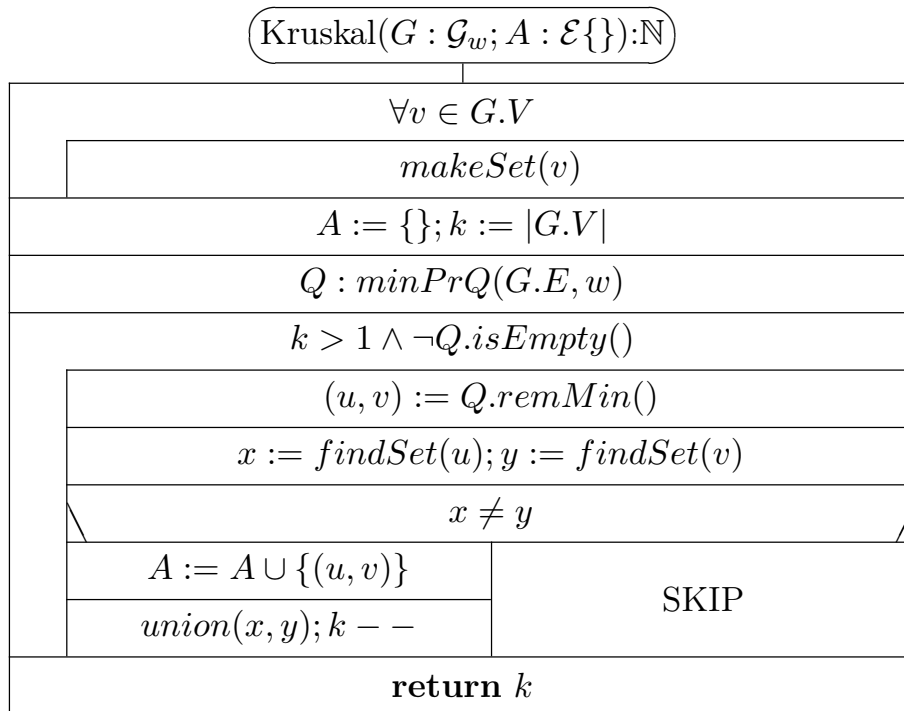


$a - b, 0$; $d, 2$; $e, 2$.
 $b - c, 1$; $e, 1$; $f, 2$.
 $c - f, 3$.
 $d - e, 0$.
 $e - f, 2$.

Components	edge	its weight	is it safe?
a, b, c, d, e, f	(a,b)	0	+
ab, c, d, e, f	(d,e)	0	+
ab, c, de, f	(b,c)	1	+
abc, de, f	(b,e)	1	+
abcde, f	(a,d)	2	-
abcde, f	(a,e)	2	-
abcde, f	(b,f)	2	+
abcdef	-	-	-



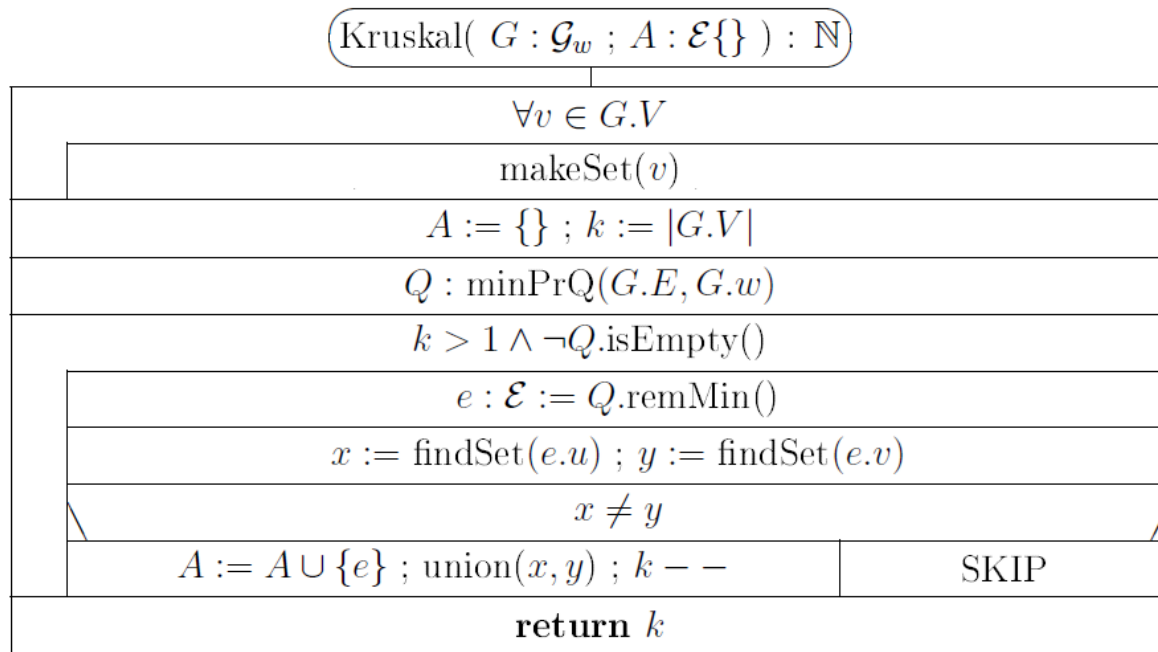
Kruskal algoritmus



Kruskal($G : \mathcal{G}_w ; A : \mathcal{E}\{\}$) : $\mathbb{N}$	
$\forall v \in G.V$	
makeSet(v) // a spanning forest of single vertices is formed	
$A := \{\}$; $k := G.V $	
// k is the number of components of the spanning forest	
// let Q be a minimum priority queue of $G.E$ by weight $G.w$:	
$Q : \text{minPrQ}(G.E, G.w)$	
$k > 1 \wedge \neg Q.\text{isEmpty}()$	
$e : \mathcal{E} := Q.\text{remMin}()$	
$x := \text{findSet}(e.u) ; y := \text{findSet}(e.v)$	
$x \neq y$	
$A := A \cup \{e\} ; \text{union}(x, y) ; k - -$	SKIP
return k	

This algorithm checks whether G is connected. If so, it returns $k = 1$. Otherwise $k > 1$.

$$MT(n, m) \in O(m * \lg n)$$



Mit csinálnak a $\text{makeSet}(v)$, $\text{findSet}(e, u)$ és $\text{union}(x, y)$ eljárások?

- Diszjunkt halmaz adatstruktúra

Diszjunkt halmaz adatstruktúra

Diszjunkt halmaz adatstruktúra

A diszjunkt halmaz adatstruktúra olyan, egymást nem metsző dinamikus halmazok gyűjteménye, ahol minden halmaznak van egy reprezentánsa, amely a halmaz egyik eleme.

$$\mathcal{S} = \{S_1, S_2, \dots, S_n\}$$

A fő műveletei:

- $\text{makeSet}(x)$: ha x nem tartozik egyetlen másik halmazhoz sem, akkor létrehoz egy új halmazt, amelynek egyetlen eleme (és így reprezentánsa) x .
- $\text{union}(x, y)$: egyesíti azt a két diszjunkt halmazt, amelyek x -et, illetve y -t tartalmazzák; a reprezentáns lehet bármelyik elem (vagy a két halmaz egyikének korábbi reprezentánsa).
- $\text{findSet}(x)$: visszaadja annak a halmaznak a reprezentánsát, amely x -et tartalmazza.

Ennek egy megvalósítása gyökeres fákkal történhet. Minden csúcs egy elemet tartalmaz, és minden fa egy halmazt reprezentál. Minden elem csak a szülőjére mutat. A fa gyökere tartalmazza a reprezentánst, és önmagára mutat.

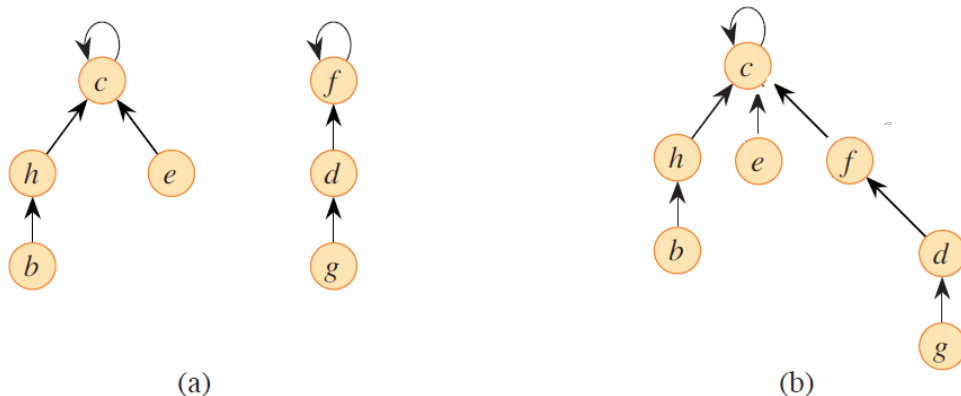
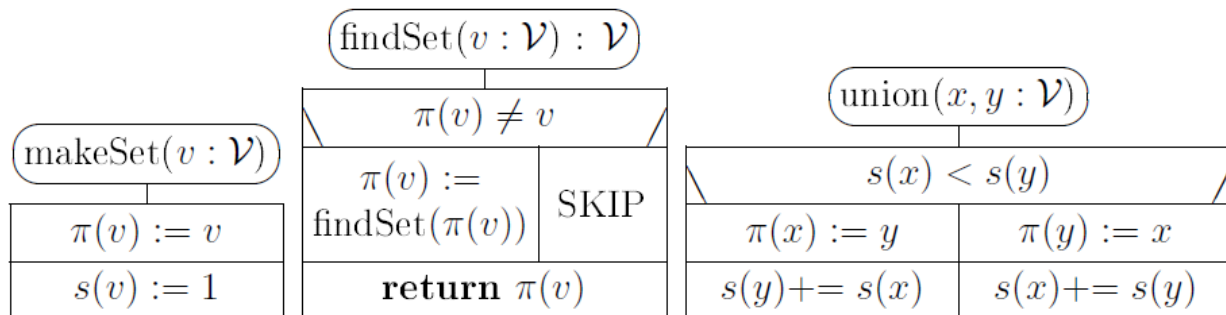


Figure 19.4 A disjoint-set forest. (a) Trees representing the two sets of Figure 19.2. The tree on the left represents the set $\{b, c, e, h\}$, with c as the representative, and the tree on the right represents the set $\{d, f, g\}$, with f as the representative. (b) The result of $\text{UNION}(e, g)$.

Minden elemnek van egy π értéke (a „szülője”), és minden halmaznak van egy s értéke (a halmaz mérete). A műveletek a következők:



- makeSet(x): létrehoz egy egyetlen elemből álló fát, amelyben x önmagára mutat.
- findSet(x): ha $\pi(x) = x$, akkor visszaadja x -et; ha nem, akkor rekurzívan meghívja önmagát $\pi(x)$ -re, és útközben frissíti $\pi(v)$ -t.
- union(x, y): a kisebb halmazt csatolja a nagyobbhoz azzal, hogy a kisebb halmaz gyökerét a másik gyökérhez kapcsolja. A méretet is módosítja.

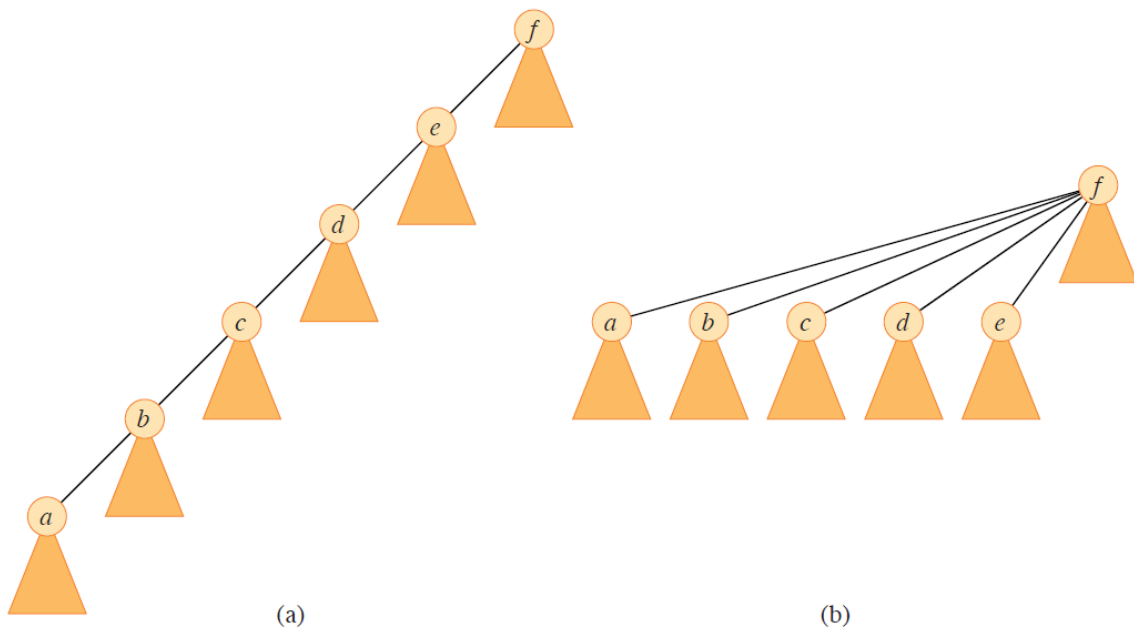


Figure 19.5 Path compression during the operation FIND-SET. Arrows and self-loops at roots are omitted. (a) A tree representing a set prior to executing FIND-SET(a). Triangles represent subtrees whose roots are the nodes shown. Each node has a pointer to its parent. (b) The same set after executing FIND-SET(a). Each node on the find path now points directly to the root.

Prim

Prim algoritmus

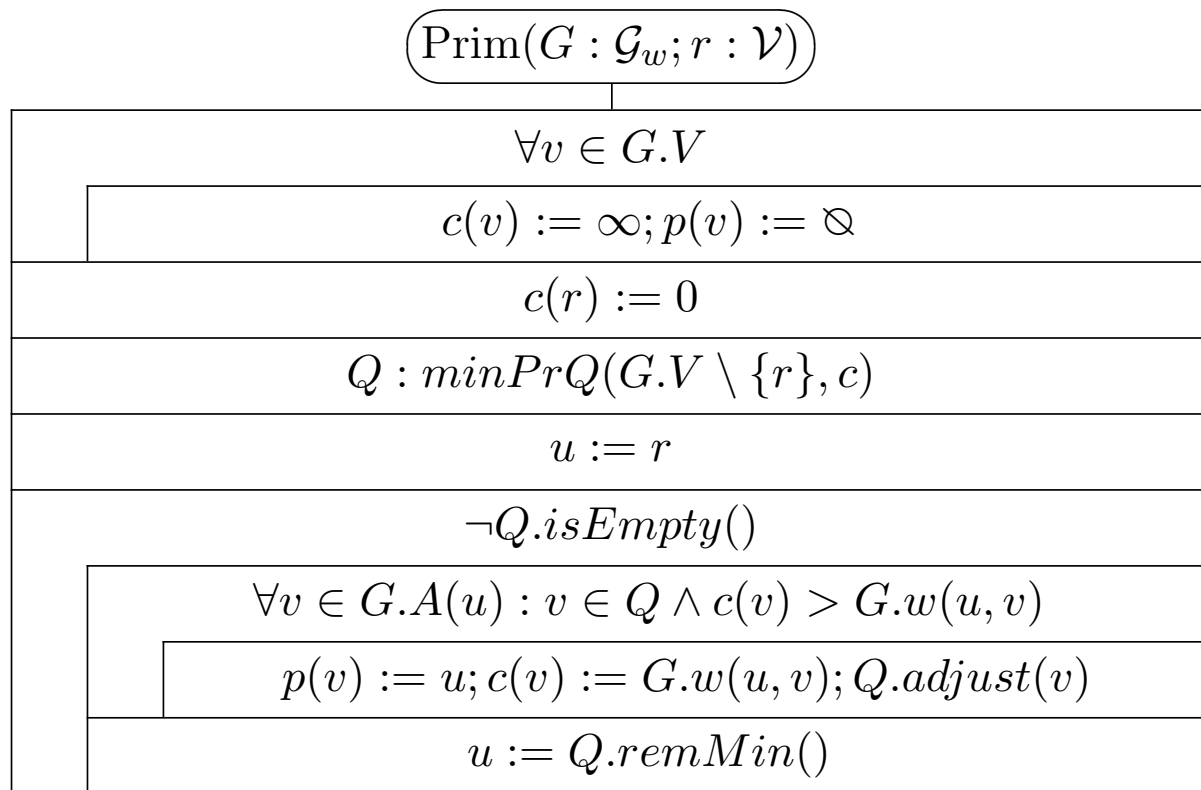
- Egy csúcsból kiindulva kezdjük el építeni a minimális feszítőfát. A csúcsokat egyenként adjuk hozzá, mindegyiket egy kapcsolódó éllel.
- Mindig azt a csúcsot választjuk, amely a legkisebb költséggel adható hozzá.
- A vágás azon csúcsok halmaza lesz, amelyeket már hozzáadtunk. Az új él egy „könnyű él” lesz ehhez a vágáshoz.
- A költségek követésére minden csúcshoz tartozik egy $c(v)$ érték (a csúcs kiválasztásának költsége), és egy minimális prioritási sort használunk a legkisebb érték kiválasztására.
- Minden v csúcshoz a $p(v)$ érték azt mutatja, melyik csúcshoz kapcsolódhat ezen a költségen.
- Amikor egy csúcsot vagy élt hozzáadunk a fához, a szomszédos csúcsok $c(v)$ és $p(v)$ értékei változhatnak, és

ezzel együtt a prioritási sor is módosul.

Prim algoritmus a minimális prioritási sorral

- Egy minimális prioritási sort használunk, hogy segítsen kiválasztani a legolcsóbb élt (és csúcsot), amit a fához kell adni.
- A költségek követéséhez minden csúcshoz $c(v)$ és $p(v)$ érték tartozik, ahol
 - $c(v)$ a csúcs kiválasztásának költsége,
 - és $(v, p(v))$ az az él, amelyhez ez a költség tartozik.
- Kezdetben minden csúcsra $p(v) = \emptyset$ és $c(v) = \infty$, kivéve a kezdőcsúcsot s , amelyre $c(s) = 0$.
- Amikor egy új csúcsot vagy élt hozzáadunk a fához, a szomszédos csúcsok $c(v)$ és $p(v)$ értékei módosulhatnak, és a prioritási sor is frissül.

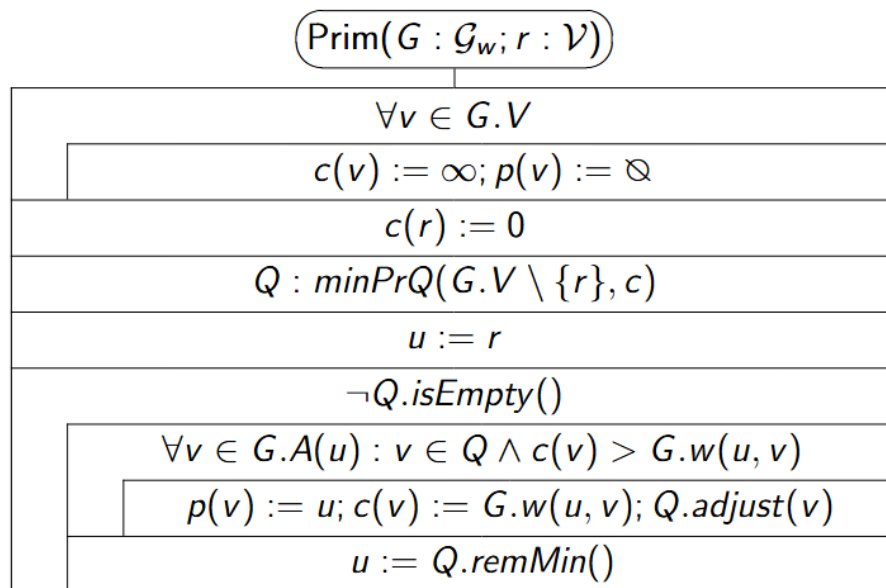
Prim algoritmusának struktogramja



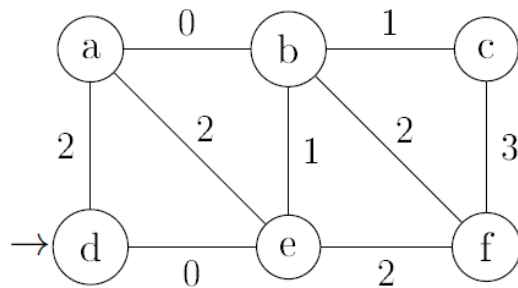
2.3 Algorithm of Prim

$\text{Prim}(G : \mathcal{G}_w ; r : \mathcal{V})$
$\forall v \in G.V$
$c(v) := \infty ; p(v) := \oslash$ // costs and parents still undefined
// edge $(p(v), v)$ will be in the MST where $c(v) = G.w(p(v), v)$
$c(r) := 0$ // r is the root of the MST where $p(r)$ remains $\oslash$
// let Q be a minimum priority queue of $G.V \setminus \{r\}$ by label values $c(v)$:
$Q : \text{minPrQ}(G.V \setminus \{r\}, c)$ // $c(v)$ = cost of light edge to (partial) MST
$u := r$ // vertex $u = r$ has become the first node of the (partial) MST
$\neg Q.\text{isEmpty}()$
// neighbors of u may have come closer to the partial MST
$\forall v \in G.A(u) : v \in Q \wedge c(v) > G.w(u, v)$
$p(v) := u ; c(v) := G.w(u, v) ; Q.\text{adjust}(v)$
$u := Q.\text{remMin}()$ // $(p(u), u)$ is a new edge of the MST

Prim algoritmusának időbonyolultsága

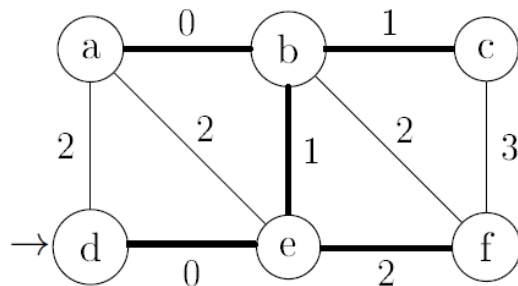


$$MT_{\text{Prim}}(n, m) = O(m \cdot \log n)$$



$a - b, 0$; $d, 2$; $e, 2$.
 $b - c, 1$; $e, 1$; $f, 2$.
 $c - f, 3$.
 $d - e, 0$.
 $e - f, 2$.

c values of nodes in Q						vertex into MST	changes of labels p					
a	b	c	d	e	f		a	b	c	d	e	f



$a - b, 0$; $d, 2$; $e, 2$.
 $b - c, 1$; $e, 1$; $f, 2$.
 $c - f, 3$.
 $d - e, 0$.
 $e - f, 2$.

c values of nodes in Q						vertex into MST	changes of labels p					
a	b	c	d	e	f		a	b	c	d	e	f
∞	∞	∞	0	∞	∞		$\otimes$	$\otimes$	$\otimes$	$\otimes$	$\otimes$	$\otimes$
2	∞	∞		0	∞	d	d				d	
2	1	∞			2	e		e				e
0		1			2	b	b		b			
		1			2	a						
					2	c						
						f						
0	1	1	0	0	2	result	b	e	b	$\otimes$	d	e

Piros-kék színezés

Forrás: https://tamop412.elte.hu/tananyagok/algorithmusok/lecke28_lap1.html

Kék szabály – piros szabály algoritmus

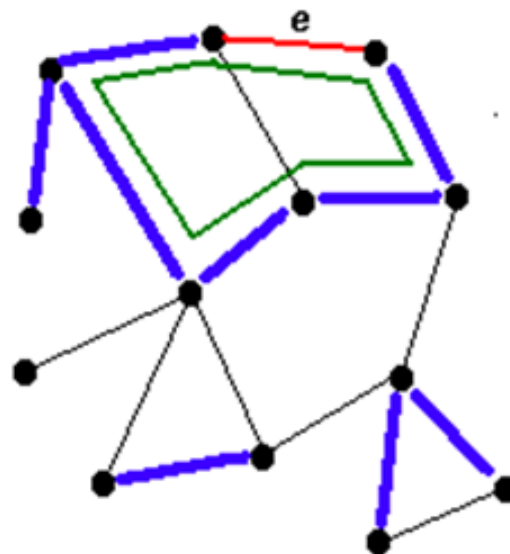
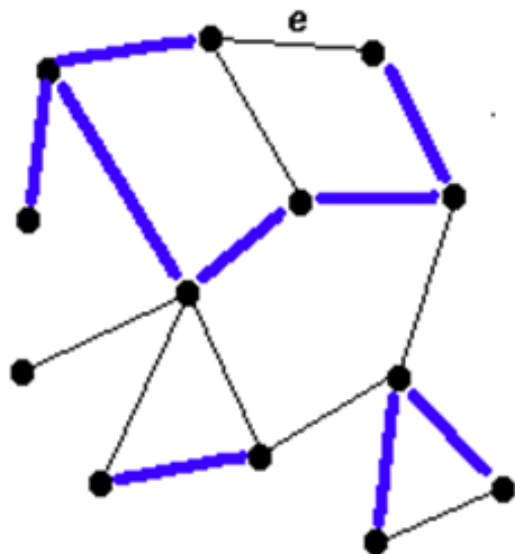
- Az algoritmus a gráf éleit színezi. Kezdetben minden él fekete.
- A kék él a minimális feszítőfa (MST) része.
- A piros él nem része az MST-nek.
- Addig folytatjuk az élek színezését, amíg $n - 1$ kék élünk nem lesz.
- Hogyan választjuk ki, melyik élt színezzük meg legközelebb, és milyen színnel?

- **Kék szabály:** Keressünk egy vágást, amely tiszteletben tartja a kék élek halmazát. Színezzük kékre a legolcsóbb fekete élt, ami átszeli a vágást.
- **Piros szabály:** Keressünk egy kört, amelyben még nincs piros él. Színezzük pirosra a kör legdrágább fekete élet.

Ezeket a szabályokat bármilyen sorrendben alkalmazhatjuk.

- A Prim algoritmusban csak a kék szabályt használjuk: a vágás mindig a már kiválasztott csúcsok halmaza.
- Az általános algoritmusban, amelyet korábban láttunk, szintén csak a kék szabályt alkalmaztuk.

Példa a kék és piros szabályok alkalmazására:

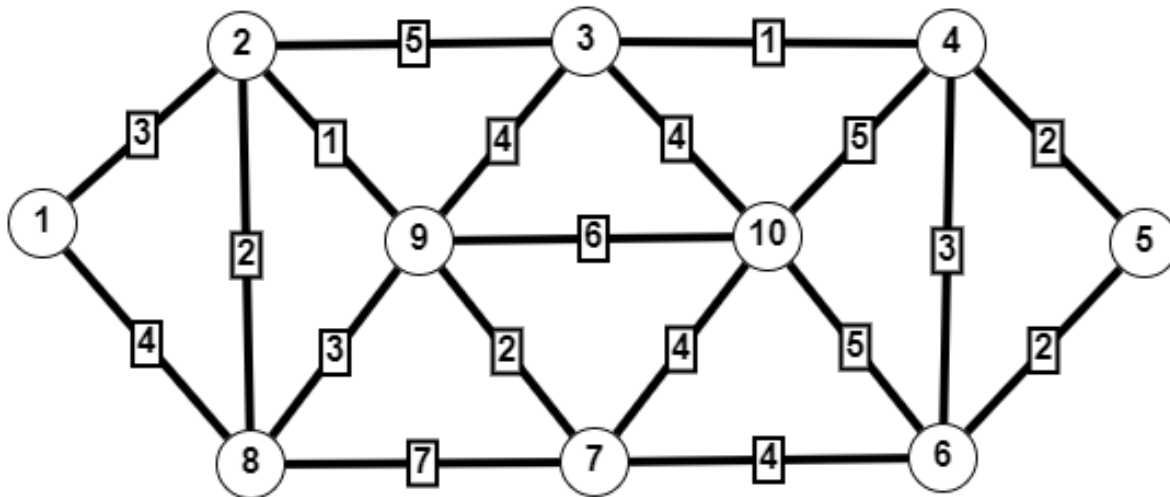


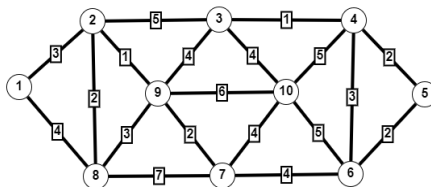
Gyakorlat

Vizualizáció:

- 1 <https://visualgo.net/en/mst>
- 2 <https://www.cs.usfca.edu/galles/visualization/Prim.html>

- 1): Mutassa be a Prim algoritmust az 1-es csúcsból kiindulva, és
 - 2): A Kruskal algoritmust az alábbi gráfon.
- Ha választási lehetősége van, mindig a kisebb indexű csúcsot, illetve az él esetében a kisebb csúcsot válassza.





$$\begin{bmatrix}
 0 & 3 & 0 & 0 & 0 & 0 & 0 & 4 & 0 & 0 \\
 3 & 0 & 5 & 0 & 0 & 0 & 0 & 2 & 1 & 0 \\
 0 & 5 & 0 & 1 & 0 & 0 & 0 & 0 & 4 & 4 \\
 0 & 0 & 1 & 0 & 2 & 3 & 0 & 0 & 0 & 5 \\
 0 & 0 & 0 & 2 & 0 & 2 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 3 & 2 & 0 & 4 & 0 & 0 & 5 \\
 0 & 0 & 0 & 0 & 0 & 4 & 0 & 7 & 2 & 4 \\
 4 & 2 & 0 & 0 & 0 & 0 & 7 & 0 & 3 & 0 \\
 1 & 0 & 4 & 0 & 0 & 0 & 2 & 3 & 0 & 6 \\
 0 & 0 & 4 & 5 & 0 & 5 & 4 & 0 & 6 & 0
 \end{bmatrix}$$

Köszönöm a figyelmet!